

CT0429 - Analisi Predittiva - aa 21/22 - Appello I

Nome Cognome - matricola

- Istruzioni
- Esercizio 1
 - Es. 1.a
 - Es. 1.b
 - Es. 1.c
 - Es. 1.d
- Esercizio 2
 - Es. 2.a
 - Es. 2.b
 - Es. 2.c
 - Es. 2.d

Istruzioni

Salvate questo file con il nome `matricola.Rmd`. Questo sarà il file che dovrete consegnare. Il file deve compilare senza problemi: files che non possono essere compilati correttamente saranno penalizzati.

Per essere sicuri di poter trovare il file al momento della consegna potete controllare dove è il file quando compilate il file la prima volta usando il comando `getwd()`.

```
getwd()
```

```
## [1] "C:/Users/TOMFAC01/Desktop/Predictive-analysis-2026/exams/2022_1"
```

```
## da cancellare quando siete sicuri di dove è il file
```

Attenzione - per tutto l'esame, se non specificato esplicitamente, il livello di significatività da usare è $\alpha = 0.02$

Esercizio 1

Il data-scientist di un'azienda che produce app ludiche per cellulari vuole costruire un modello predittivo per il tempo in cui gli utenti hanno usato l'app. Dal dataset disponibile estrae i records delle persone che, nel periodo di Gennaio-Marzo 2021 hanno dichiarato al momento dell'iscrizione di voler condividere con l'azienda l'informazione sulla loro età: questo identifica 70 utenti per cui sono disponibili le seguenti informazioni:

- `numAccess` : il numero totale di accessi eseguiti all'app dall'utente nell'anno 2021
- `age` : l'età dichiarata dall'utente
- `time` : il tempo passato dall'utente usando l'app

La variabile `time` è la variabile di interesse (variabile risposta).

Tutte le informazioni a disposizione dell'analista sono disponibili nel dataset `dex1` che si può caricare usando il seguente codice:

```
dex1 <- read.csv("ex1_data.csv", header = TRUE)
```

Es. 1.a

```
fit1 <- lm(time~., data = dex1)
summary(fit1)
```

```
##
## Call:
## lm(formula = time ~ ., data = dex1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -515.80 -111.54    8.45  135.21  874.33
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 113.0697   102.7900   1.100   0.275
## numAccess   24.8515    0.8375  29.673 <2e-16 ***
## age         0.2045     2.2421   0.091   0.928
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 243 on 67 degrees of freedom
## Multiple R-squared:  0.9293, Adjusted R-squared:  0.9272
## F-statistic: 440.3 on 2 and 67 DF,  p-value: < 2.2e-16
```

Il modello è altamente significativo e il valore di R^2 è molto alto: il modello spiega una ampia proporzione della variabilità nei dati. Si nota inoltre che solo una delle variabili esplicative risulta significativa.

Es. 1.b

```
fit2 <- lm(time ~ numAccess, data = dex1)
anova(fit2, fit1)
```

```
## Analysis of Variance Table
##
## Model 1: time ~ numAccess
## Model 2: time ~ numAccess + age
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1      68 3957522
## 2      67 3957031  1    491.29 0.0083 0.9276
```

Usando anova per confrontare fit1 e fit2 (che sono due modelli annidati) si verifica l'ipotesi nulla:

$$H_0 : \beta_{age} = 0$$

contro l'ipotesi alternativa

$$H_1 : \beta_{age} \neq 0$$

Si verifica cioè se è possibile eliminare un predittore dal modello (e stimare un parametro in meno) ed ottenere una stima comparabile in termine di errore quadratico medio (RSS). Dato che il p-value del test `anova` è molto alto non si può rifiutare l'ipotesi nulla: non vi è evidenza che mantenere la variabile `age` migliori la stima del modello e possiamo quindi eliminare la variabile.

Es. 1.c

```
cis <- predict(fit2,
               newdata = data.frame(numAccess = c(60, 200)),
               interval = "confidence")
cis[,3]-cis[,2]
```

```
##          1          2
## 132.4136 542.3645
```

```
pis <- predict(fit2,
               newdata = data.frame(numAccess = c(60, 200)),
               interval = "prediction")
pis[,3]-pis[,2]
```

```
##          1          2
## 971.8542 1105.0459
```

```
summary(dex1$numAccess)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      6.00   18.00   27.50   40.26   50.75  157.00
```

Il secondo intervallo di confidenza è più ampio del primo: questo perché non ci sono utenti nel campione che abbiano eseguito 200 accessi: per fare una stima per quel tipo di utente è necessario estrapolare la stima ben oltre i valori con cui il modello è stato stimato e quindi vi è molta incertezza nella stima. Questo è vero sia per gli intervalli di confidenza che per gli intervalli di predizione.

Gli intervalli di confidenza danno un'indicazione dell'incertezza attorno al valore medio del tempo passato da ogni utente nell'app; gli intervalli di predizione invece danno un'indicazione dell'incertezza per quelli che possono essere gli effettivi valori del tempo passato da un utente nell'app. Dato che la media è per definizione meno variabile delle singole osservazioni gli intervalli di confidenza sono meno ampi degli intervalli di predizione.

Es. 1.d

Il data scientist ha incluso nell'analisi solo le persone che hanno accettato di condividere con l'azienda l'informazione sulla loro età. Questo gruppo di persone potrebbe non essere rappresentativo della popolazione generale di utenti dell'app, dato che sono probabilmente persone con una alta fidelizzazione nei confronti dell'app. Per poter generalizzare i risultati sarebbe necessario poter essere sicuri che gli utenti che consentono a condividere l'informazione sull'età non si comportino diversamente dagli altri utenti.

Esercizio 2

Il gestore di un sistema di rete mantiene un registro di informazioni varie in cui registra anche se in una giornata si sono registrati guasti che richiedono l'intervento di un operatore esterno per ripristinare il sistema. Il file `ex2_data.csv` contiene le seguenti variabili:

- `numAccess` : il numero di accessi alla rete
- `numRequests` : il numero di richieste gestite dalla rete
- `durationMean` : la durata media di una sessione di un utente
- `durationMax` : la durata massima di una sessione di un utente
- `durationMin` : la durata minima di una sessione di un utente

- `fail` : un indicatore che ha valore 1 quando si è registrato un guasto che richiede l'intervento di un operatore esterno per ripristinare il sistema

Si carichi il dataset usando il codice seguente:

```
dex2 <- read.csv("ex2_data.csv", header = TRUE)
```

Si desidera costruire un modello predittivo per la variabile `fail`, un modello cioè che predica se in un giorno sarà necessario l'intervento di un operatore esterno per ripristinare il sistema.

Es. 2.a

```
fitTot <- glm(fail ~ ., family = binomial, data = dex2)
summary(fitTot)
```

```
##
## Call:
## glm(formula = fail ~ ., family = binomial, data = dex2)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -5.901e+00  1.805e+00  -3.269 0.001080 **
## numAccess    7.528e-04  7.142e-05  10.541  < 2e-16 ***
## numRequests -1.728e-04  1.086e-04  -1.591 0.111690
## durationMean  6.200e-02  1.610e-02   3.851 0.000118 ***
## durationMax   6.959e-03  1.872e-02   0.372 0.710041
## durationMin   3.834e-02  4.791e-02   0.800 0.423539
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 619.12  on 449  degrees of freedom
## Residual deviance: 369.09  on 444  degrees of freedom
## AIC: 381.09
##
## Number of Fisher Scoring iterations: 5
```

La stima dei parametri che descrivono l'effetto di `numAccess` e `numRequests` è di segno opposto: si stima che al crescere del numero di accessi aumenta la probabilità che si verifichi un guasto, mentre questa probabilità diminuisce all'aumentare del numero di richieste. Quest'ultimo effetto però viene stimato essere poco significativo (quindi non si può rifiutare $H_0 : \beta_{numRequests} = 0$).

Es. 2.b

```
print(step(fitTot, direction = "backward", trace = 0, k = 2))
```

```
##
## Call:  glm(formula = fail ~ numAccess + numRequests + durationMean,
##         family = binomial, data = dex2)
##
## Coefficients:
## (Intercept)      numAccess    numRequests    durationMean
## -5.0861333      0.0007530     -0.0001813      0.0637769
##
## Degrees of Freedom: 449 Total (i.e. Null);  446 Residual
## Null Deviance:      619.1
## Residual Deviance: 369.9      AIC: 377.9
```

```
print(step(fitTot, direction = "backward", trace = 0, k = log(450)))
```

```
##
## Call:  glm(formula = fail ~ numAccess + durationMean, family = binomial,
##         data = dex2)
##
## Coefficients:
## (Intercept)      numAccess    durationMean
## -5.7172950      0.0007083      0.0618555
##
## Degrees of Freedom: 449 Total (i.e. Null);  447 Residual
## Null Deviance:      619.1
## Residual Deviance: 372.7      AIC: 378.7
```

Usando il BIC come criterio per selezionare il modello ottimale si individua un modello che ha meno parametri rispetto a quello individuato usando AIC come criterio. Questo deriva dal fatto che BIC penalizza maggiormente i modelli con molti parametri (specialmente quando $\log(n)$ è molto più grande di 2) e tende quindi a identificare come ottimali modelli con meno parametri rispetto ad AIC.

Es. 2.c

```
nd <- data.frame(numAccess = c(10,5290),
                 numRequests = c(2000,5000),
                 durationMean = c(10,24.8),
                 durationMax = c(75,90),
                 durationMin = c(9,5))
rownames(nd) <- c("day1","day2")
```

```
predict(fitTot, type = "response", newdata = nd)
```

```
##          day1          day2
## 0.008557813 0.394756062
```

```
preds <- predict(fitTot, type = "link", newdata = nd, se.fit = TRUE)
cbind(binomial()$linkinv(preds$fit + qnorm(0.025)*preds$se.fit),
      binomial()$linkinv(preds$fit + qnorm(0.975)*preds$se.fit))
```

```
##          [,1]      [,2]
## day1 0.002655152 0.02722447
## day2 0.333659345 0.45932840
```

Es. 2.d

```
## proporzione predizione corrette  
mean(as.numeric(predict(fitTot, type = "response") > 0.5) == dex2$fail)
```

```
## [1] 0.7844444
```

```
## proporzione predizioni sbagliate  
mean(as.numeric(predict(fitTot, type = "response") > 0.5) != dex2$fail)
```

```
## [1] 0.2155556
```

```
## tabella di contingenza di predizioni corrette e sbagliate  
table(as.numeric(predict(fitTot, type = "response") > 0.5), dex2$fail)
```

```
##  
##      0    1  
## 0 200  49  
## 1   48 153
```

Il modello predice in maniera corretta l'evento `guasto` in circa il 78.4 % dei casi.